

面向 AI 原生系统的智能运维

第 10 讲 | 大型客户智能运维转型实践

从工具试点到数据、平台、流程、组织和持续运营的能力体系

陈鹏飞 | 中山大学

从第 9 讲继续：一个 Agent 如何变成企业能力？



第 9 讲 workflow、证据、
状态、审批与回滚。

本讲 大型客户如何选
择、治理和规模化。

后续章节 平台架构、
Agent 开发与专题实验。

本讲学习目标

1. 解释大型客户智能运维转型的系统性难点；
2. 从价值、风险、数据和接管成本选择首个场景；
3. 设计数据治理、知识、工具、流程和平台的最小闭环；
4. 理解从人工运维到 AIOps、LLMOps、AgentOps 的能力演进；
5. 为试点建立阶段门、基线、验收指标和停止条件；
6. 讨论平台团队、业务 SRE、治理委员会和持续运营机制。

最终能力：给出一条“为什么现在做、先做什么、如何证明有效、如何安全扩大”的路线。

开场案例：大促前，系统真的准备好了吗？

客户现状

- ▶ 上千微服务、数千容器实例；
- ▶ 峰值流量和业务流程快速变化；
- ▶ 指标、日志、Trace 由不同团队维护；
- ▶ 发布采用蓝绿/金丝雀，但排障仍靠人工。

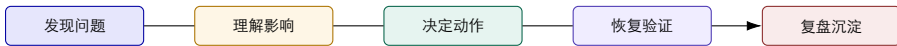
大促当天

- ▶ 缓存热 Key 失效，连接池打满；
- ▶ 购物车 RT 从 5ms 升到 8s；
- ▶ 重试与消息堆积扩散至交易链路；
- ▶ 下单成功率从 99.8% 降到 12%，定位耗时 53 分钟。

转型问题 客户缺的不是一句“请生成 RCA”，而是上线前发现脆弱点、跨源取证、可回放演练和受控处置的体系。

来源：脱敏电商故障案例；数字为单次案例快照，不作为行业统计。

什么叫“智能运维转型”？



- ▶ 从“人找数据”转为“数据和事件可关联”；
- ▶ 从“人记经验”转为“知识有版本、可复用、可复审”；
- ▶ 从“人执行命令”转为“流程有门禁、动作可回滚”；
- ▶ 从“一次性项目”转为“有指标、有反馈、有预算的运营能力”。

2 学时路线图

1. 大型客户为什么难：复杂系统、数据断点、组织断点；
2. 能力如何演进：人工、监控、AIOps、LLMOps、AgentOps；
3. 转型如何启动：场景组合、阶段门、最小闭环；
4. 平台如何支撑：数据、知识、工具、流程、治理；
5. 组织如何运营：平台团队、业务团队、CoE、复盘和预算；
6. 案例研讨：大促级联故障、告警治理、智能体平台化。

一、大型客户为什么难

复杂系统把技术问题放大为数据、流程和责任问题

大型客户的“复杂”不是只有规模大

系统 微服务、容器、GPU、模型、数据和网络跨层。

业务 交易、风控、客服、营销等流程相互依赖。

变化 版本、流量、策略和组织边界持续变化。

责任 多个团队共同拥有一条用户请求链。

核心判断 规模把局部缺陷变成级联风险；组织把局部修复变成跨团队协调。

五行业调研：问题表现不同，断点高度相似

 2025年10月，AWS DynamoDB 全球宕机**14小时**，云原生架构的**深度依赖链放大效应**使**上百个**依赖服务（如 EC2、S3、Lambda）连环失效，超 **3500 家**公司、**60 + 国家**受影响，用户报告超 **1700 万**次，仅亚马逊自身损失就达 **20 亿美元** (<https://aws.amazon.com/cn/message/101925/>)

 2025年6月，Google Cloud全球宕机**8小时**，云原生基础设施的分布式状态同步机制失效，导致全球网络路由表错误，受影响服务超过**60项**，涵盖Gmail、Google Drive、App Engine、Cloud Functions，全球电商交易损失约**45 亿美元**，金融市场交易暂停 **120 分钟** (<https://www.51cto.com/article/818413.html>)

 2023年，国内某云厂商**API调用异常**导致大量关联App崩溃，**持续时间4小时**

 2023年，Office 365在线应用和 Teams协作出现问题，导致**服务 Office 365 不可用持续近6小时**

 2024年，某省通信运营商由于内部网络错误导致核心业务办理不成功，**排查时间长达4天**

 2021年，由于配置变更不当，导致数据中心间通信中断，**宕机长达7个小时**

 2023年，某出行App由于底层软件故障导致**应用功能瘫痪持续近12小时**

 2023年，由于**人为运维操作错误**，导致亚马逊S3服务**中断长达5小时**，大面积云服务应用无法使用

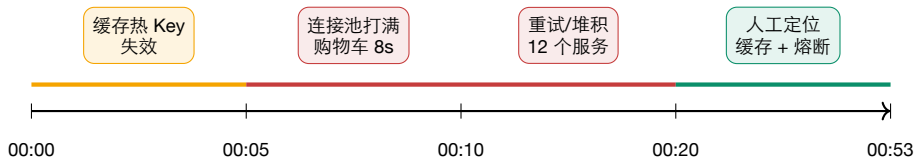
- ▶ 金融：历史系统迁移、依赖梳理和架构成熟度；
- ▶ 电力：预测误报、经验库缺失和复杂故障恢复；
- ▶ 制造：国产化数据格式不统一、跨层定因困难；
- ▶ 通信：告警量大、重复高、工具编排需求强；
- ▶ 电商：高峰流量、频繁变更、演练覆盖和级联风险。

来源：脱敏企业调研案例；数字仅用于课堂场景比较。

把客户痛点归并为五类断点

断点	表现	转型回应
数据断点 知识断点 工具断点 流程断点 责任断点	指标、日志、Trace、工单分属不同系统 Runbook、历史事故和专家经验不可复用 每个团队维护脚本，接口、权限和失败语义不同 告警、工单、审批、执行、验证互相脱节 平台、应用、网络、数据团队边界不清	统一身份、时间水位、查询和血缘 版本化知识、规则、案例和复审 Tool Registry、契约、评分和淘汰 事件 ID、状态机、阶段门和回滚 角色、接管、SLO 和治理机制

电商案例的级联时间线



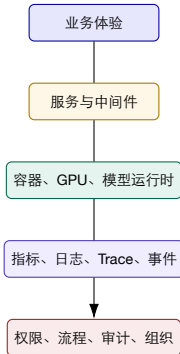
教学解读 转型优先级应落在“变更前检查 + 缓存/熔断可观测性 + 演练覆盖 + 证据驱动 RCA”，而不是只在事故后生成报告。

为什么故障会从一个组件扩散到整条交易链？



- ▶ 超时、重试、线程池和消息队列会把局部慢放大为系统级拥塞；
- ▶ 没有调用链、队列、依赖和变更的联合视图，就无法判断传播方向；
- ▶ 转型把“事后定位”前移到依赖建模、韧性测试和变更治理。

系统边界：客户买的不是一个监控面板



每一层都应能回答：谁负责、看什么、怎样验证、出现风险时如何接管。

运维人员的真实旅程：20 分钟花在“找数据”

1. 从告警消息复制服务名和时间；
2. 切到指标平台确认是否真的异常；
3. 再切日志平台搜索关键词；
4. 找到 Trace 后手工对齐调用链；
5. 在群聊里询问版本、变更和依赖负责人；
6. 最后才开始判断根因和动作。

转型切入 第一阶段的价值往往不是“模型替工程师诊断”，而是让工程师少做跨平台搬运和上下文拼接。

四个常见误区

买模型 模型接入
不等于生产能力。

先做大平台 没有
场景和验收，平台
变成组件仓库。

只报准确率 忽略
成本、安全、接管
和数据漂移。

自动化越多越好
高风险动作没有回
滚和责任人。

转型的第一问不是“用哪个模型”，而是“哪个责任链现在最值得被重构”。

转型的四个杠杆



闭环 数据支撑流程，流程约束平台，平台降低重复劳动，人反馈新的数据和规则。

价值链：从发现事件到组织学习

发现

减少漏报

理解

减少排障

处置

减少恢复

验证

避免复发

学习

提高下一次

- ▶ 每个环节都有不同数据、角色和指标；
- ▶ 只做“告警降噪”无法证明业务恢复变快；
- ▶ 只做“自动修复”如果没有验证和复盘，可能放大风险。

目标状态：运维能力是一条可观察的链



平台化的关键是所有节点都留下事件、版本、责任和证据，而不是只保留最终答案。

转型前先问三个问题

1. 哪个业务损失或工程瓶颈最真实、最频繁、最可量化？
2. 哪些数据和工具已经可用，哪些必须先补齐？
3. 谁会在真实值班、变更或事故中接管它？

反例 如果回答是“所有团队都需要”“数据以后再说”“先由模型自动处理”，说明还没有形成可执行的转型问题。

第一部分小结：把“复杂”拆成可治理的断点

1. 复杂系统放大级联风险；
2. 多团队和多平台制造数据、知识、工具、流程、责任断点；
3. 转型对象是责任链，不是单个模型；
4. 首先要建立可观测、可验证、可接管的最小闭环。

二、能力如何演进

从手工操作到 AIOps、LLMOps 与 AgentOps

阶段一：手工运维为什么仍然存在？

它解决了什么

- ▶ 复杂异常由专家临场判断；
- ▶ 新系统没有历史数据也能处理；
- ▶ 动作和责任在人的控制范围内。

它的代价

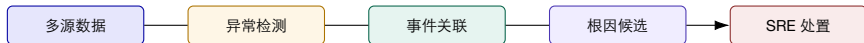
- ▶ 依赖少数专家，难以跨班次复制；
- ▶ 经验、命令、证据和复盘分散；
- ▶ MTTR 高，演练和预防常被挤掉。

阶段二：监控与告警——看见，不代表理解



- ▶ 解决“是否异常”，但不一定解决“为什么”和“怎么办”；
- ▶ 阈值规则对动态流量、版本和多维属性不稳定；
- ▶ 告警量增加后，人的注意力成为瓶颈。

阶段三：AIOps——把检测、关联和预测接起来



AIOps 的重要变化是从“单指标告警”走向“多源事件与故障上下文”，但工具和流程仍可能是人工拼装的。

阶段四：LLMOps——把知识和交互带进运维

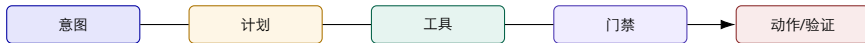
新增能力

- ▶ 用自然语言访问多类数据；
- ▶ 对日志、工单、Runbook 做语义检索；
- ▶ 生成摘要、解释、查询和操作草稿。

新增风险

- ▶ 数据泄露和越权检索；
- ▶ 幻觉、过期知识和错误引用；
- ▶ 成本、延迟和模型版本漂移。

阶段五：AgentOps——把模型输出变成受控流程

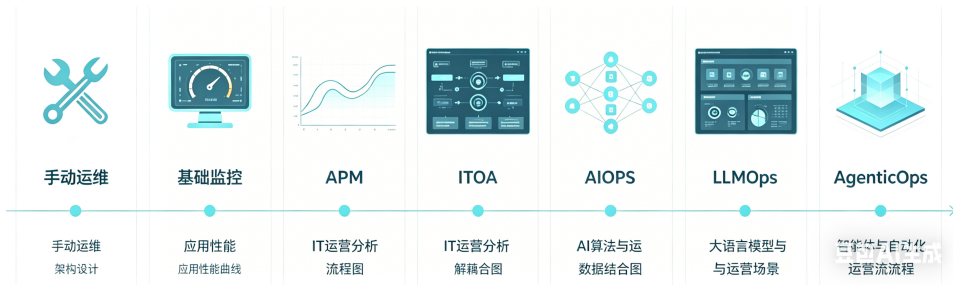


- ▶ 需要轨迹、状态、权限、审批、回滚和评测；
- ▶ AgentOps 的对象是“任务执行系统”，不是一个 Prompt 版本；
- ▶ 第 9 讲讲 workflow 机制，本讲讲企业如何把它运营起来。

五个阶段的对照

阶段	主要对象	主要收益	关键短板
手工 监控 AIOps LLMOps AgentOps	人与命令 指标与告警 多源事件与模型 知识、语义交互 任务、工具、审批、轨迹	灵活、可处理未知 看见问题 关联、预测、RCA 检索、解释、摘要 参与执行、可复盘	难复制、难度量 噪声多、缺上下文 工具/流程碎片化 幻觉、治理、成本 安全、责任、运营复杂

运维能力演进：自动化提高，治理要求同步提高



阅读这张条带时要同时看两条轴：智能化程度增加，组织和治理要求也同步增加。

来源：脱敏企业案例中的独立演进图。

大模型应该放在哪一层？

感知层 解析指标、日志、事件和工单；不直接下结论。

认知层 生成假设、计划、摘要和解释；必须绑定证据。

执行层 通过工具、策略和审批改变系统；不接受自由文本直通。

边界 模型适合处理语义和不确定性，规则/状态机适合处理权限、终态和不可违反的不变量。

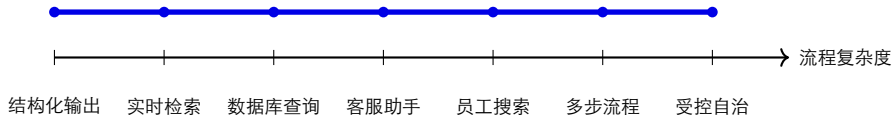
企业生成式 AI 的四个平台成功因素



- ▶ 模型选择：按任务、质量、延迟和预算组合；
- ▶ 数据定制：检索、规则、反馈和领域知识；
- ▶ 生产管理：部署、版本、观测、成本和回滚；
- ▶ 灵活性：避免把所有责任锁在单一供应商或单一接口上。

来源：《Enterprise Trends for Generative AI》相关企业生成式 AI 平台成功因素；本页为教学重绘。

Function Calling 的梯度：从结构化输出到自主 workflows



越向右，越需要流程状态、权限、验证器和人工接管；不是只增加一个工具调用。

来源：《Enterprise Trends for Generative AI》Function Calling 相关页面；教学重绘。

转型不是替代人员，而是重分配注意力

机器做得好 检索、聚合、重复检查、轨迹记录、候选排序。

人做得好 业务影响判断、模糊异常、风险权衡、跨团队协调。

共同完成 假设验证、动作审批、恢复确认、知识复审。

课程思政 关键基础设施不能只追求“自动化更多”，还要追求责任清晰、风险可控和工程质量可持续。

第二部分小结：能力演进也意味着治理升级

1. 从手工到监控，解决可见性；
2. 从监控到 AIOps，解决事件关联和预测；
3. 从 LLMOps 到 AgentOps，解决知识交互和受控执行；
4. 每一步能力升级，都同时增加数据、权限、评测和组织要求。

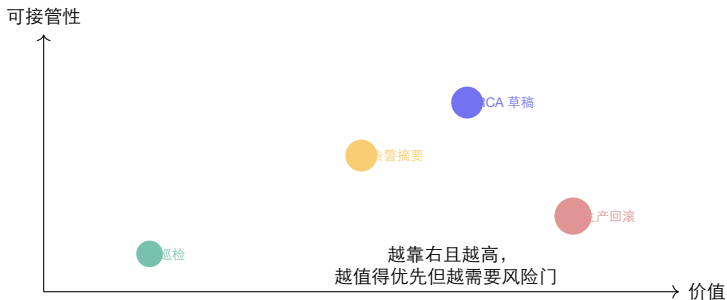
三、如何启动转型

先选对场景，再建立能被验收的最小闭环

场景组合：不要把所有痛点同时开工

场景	价值	风险	首轮建议
只读巡检	中高：减少日常人工检查	低	适合作为第一批
告警归并/摘要	高：减少噪声和上下文切换	中	需要历史标签和人工反馈
RCA 草稿	高：缩短排障时间	中高	需证据绑定和人工复核
低风险动作	中高：减少重复操作	中高	只开放幂等、可回滚动作
生产回滚/删资源	高但偶发	高	不适合作为首个试点

四维场景评分：价值、风险、数据、接管



数据可用性和接管成本作为额外门槛；高价值不自动等于高优先级。

阶段门 1：问题价值是否真实？

要有基线

- ▶ 过去 4 周告警量、有效率和 MTTR；
- ▶ 人工排障时间和跨团队次数；
- ▶ 变更失败率、业务不可用时长；
- ▶ 现有 Runbook 覆盖率。

不通过时

- ▶ 重新定义问题和影响对象；
- ▶ 只做观测/数据质量试点；
- ▶ 不以“模型很强”替代业务价值证据。

阶段门 2：数据与工具是否可用？

检查项	合格表现	不合格补救
身份	service、cluster、owner 可关联	先建对象目录和映射表
时间	指标、日志、Trace 有共同水位	校正时钟、记录采样和延迟
语义	错误、告警、事件可分类	建字段规范、标签和词表
权限	查询/变更范围可审计	只开放只读或沙箱
工具	schema、失败语义、回滚和验证齐全	封装 Tool Registry 或暂不自动调用

阶段门 3：流程与责任是否可接管？

1. 谁触发任务、谁看结果、谁批准动作？
2. 谁在夜间、节假日和跨团队场景接管？
3. 什么结果算完成，什么结果必须升级？
4. 人接管后，系统是否保留状态、证据和下一步？
5. 发生误报、漏报、误动作时，谁负责复盘？

原则 没有明确接管人的自动化，只是把风险从一个队列转移到另一个队列。

阶段门 4：结果是否能持续运营？

质量 有回归集、人工复核和失败分类。

成本 有 Token、工具调用、基础设施和人力预算。

治理 有版本、灰度、审计、过期和回滚。

通过一次 Demo 只能进入试运行，不能直接进入规模化。

适合首个试点的“低风险高频”场景

每日巡检 固定清单、只读查询、异常摘要、负责人分派。

告警治理 归并重复事件、补充上下文、生成摘要、人工确认规则。

变更前检查 检查依赖、容量、版本、回滚和维护窗口。

为什么 这些场景频率高、结果可验证、动作可先保持只读，便于建立信任和数据闭环。

不适合作为第一个试点的三类场景

高风险写操作 删库、删命名空间、无回滚数据修复。

低频黑天鹅 没有历史样本，收益和误差难以估计。

无人接管 跨部门、无明确 Owner、无法在夜间响应。

这些场景可以进入仿真、dry-run 或演练，但不应作为首个生产自治目标。

试点章程：一页写清楚再开工

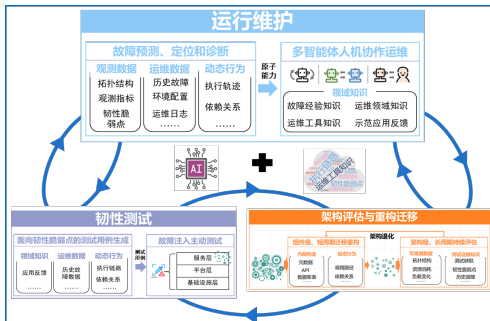
字段	需要写清
目标	例如：把跨平台告警分诊时间从 20 分钟降到 8 分钟以内
范围	服务、集群、班次、时间窗、数据源和用户角色
不做什么	不执行生产写操作，不替代事故 Commander
输入/输出	告警事件、证据链接、候选排序、工单草稿
验收	任务成功、证据覆盖、安全、成本、采用度
停止	数据质量下降、误动作、成本超预算、无人接管

最小闭环：六个步骤即可启动



第一个闭环可以没有自动修复，但不能没有验证、复盘和下一轮改进。

维护—测试—知识反馈循环



把运行中发现的问题带回韧性测试，把验证后的结论沉淀为知识，再反哺下一轮运行维护。

来源：脱敏企业案例中的独立反馈循环图。

案例 A：电商大促转型路线

1. 变更前：缓存、熔断、连接池、队列和依赖检查；
2. 演练中：主动注入缓存热 Key 失效，观察扩散路径；
3. 运行中：提前预警、证据驱动 RCA、最小动作建议；
4. 恢复后：自动生成时间线、根因、缺口和回归任务；
5. 下次大促：用同一故障 ID 复演，比较是否真的变好。

转型逻辑 把一次事故从“损失”变成“测试样本、知识条目、变更门禁和下一次验收”。

案例 A：三类验收证据

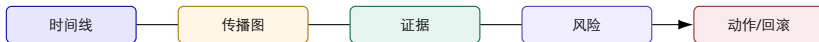
预防证据 变更前发现缓存/熔断风险；故障注入覆盖目标路径。

诊断证据 缓存、连接池、RT、重试、队列和调用链形成时间线。

恢复证据 业务成功率、SLO、队列、错误率和回滚结果均恢复。

只展示“定位到了缓存”不够；要证明提前发现、减少影响、恢复有效并且知识可复用。

案例 A：平台不应只展示一个 RCA 文本



报告只是最终视图；平台应能下钻到事件、查询、工具调用、审批和恢复验证。

案例 B：告警治理的正确顺序

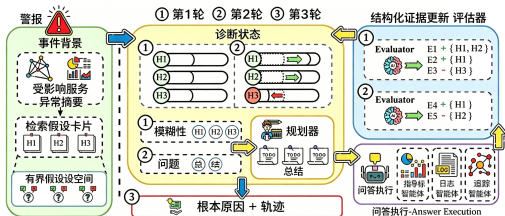
1. 统一事件身份：服务、对象、告警类型、时间水位；
2. 识别重复和关联：规则、拓扑、历史和时间窗口；
3. 生成摘要：影响面、证据、候选和下一步；
4. 人工反馈：合并是否正确、是否漏掉关键事件；
5. 离线规则迭代：版本、灰度、回归和回滚。

警告 没有身份和时间线，直接用 LLM 做告警摘要，往往只是把噪声写得更顺。

案例 B：告警治理如何从“少一点”变成“更可信”

阶段	观察指标	业务解释
去噪	重复率、有效率、漏报抽样	值班人是否少看无效事件
归并	事件数量、拓扑覆盖、时间线完整度	是否把同一事故聚在一起
摘要	证据覆盖、人工采纳、错误引用	是否帮助下一步判断
规则演化	误报/漏报变化、回归失败	是否长期稳定而非短期下降

案例 C: 从 OpsLoop 到可复用评测资产



- ▶ 故障场景、注入真值、告警、诊断和恢复一起保存;
- ▶ History 让一次运行成为可回放样本;
- ▶ Knowledge 记录规则和演化版本;
- ▶ OpsEval 让不同流程、知识切片和模型可比较。

来源: OpsEval 原型中的独立评测闭环图; 属于阶段性设计。

从试点到规模化：四道阶段门



门不通过时，允许降级为只读、仿真、单集群或单班次，而不是硬推规模化。

规模化策略：横向复用，不是复制一套 Demo

统一 身份、事件、
工具、权限、审计。

配置 服务、集群、
数据源和风险参数
可配置。

扩展 技能、规则、
查询和流程可插拔。

比较 相同真值、
版本和预算下可复
测。

规模化判断 如果每个新团队都要重新写采集、工具、审批和评测，平台还没有形成复用能力。

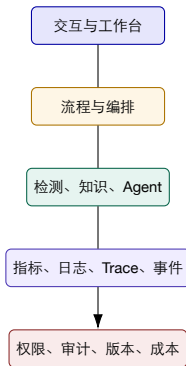
第三部分小结：先闭环，再扩面

1. 用价值、风险、数据和接管成本筛选场景；
2. 用基线、验收、停止条件写试点章程；
3. 用维护—测试—知识反馈把一次运行变成长期资产；
4. 用阶段门决定继续、降级、补数或停止。

四、平台与数据治理

平台不是组件堆，而是跨团队责任链的共同语言

平台的能力分层



治理不是最底层的“附加模块”，它贯穿每一层：谁能看、谁能做、如何记录、如何回滚。

数据底座：先把“同一个对象”认出来

统一对象身份

- ▶ service、workload、pod、node、cluster；
- ▶ owner、环境、区域、版本、业务域；
- ▶ 变更单、告警、Trace、工单关联。

没有统一身份会怎样

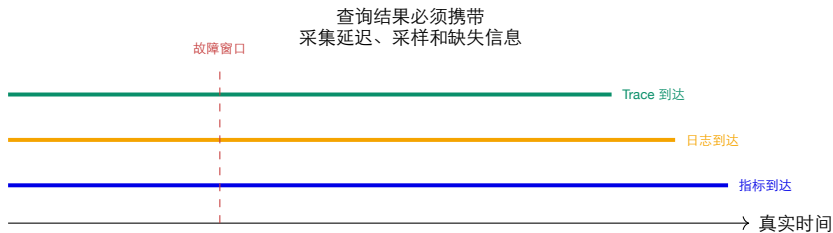
- ▶ 同一服务被不同平台命名；
- ▶ 查询结果无法合并；
- ▶ 责任人和影响面无法自动推断；
- ▶ Agent 只能让人再次手工确认。

数据契约：跨团队合作的最低共同语言

字段	例子	作用
对象	service=checkout, cluster=prod-cn	定义查询与权限范围
时间	observed_at、window、watermark	对齐多源数据
语义	signal_type、severity、event_class	归并与路由
血缘	source、query_id、transform	可复核、可审计
质量	freshness、missing、partial、confidence	防止模型误读不完整数据

数据契约不是把所有字段设计得很复杂，而是先把高频任务需要的最小字段固定下来。

时间水位：跨平台关联的隐藏难题



工程问题 没有时间水位，Agent 可能把恢复后的正常数据和故障时的异常证据混在一起。

数据质量指标：质量本身也需要观测

新鲜度

延迟多久

完整度

缺失多少

一致性

跨源是否对齐

可用性

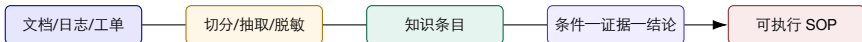
查询是否成功

可信度

是否人工确认

数据质量下降时，系统应降低自治等级或明确输出“当前无法判断”，而不是继续生成确定结论。

知识治理：文档不是知识库，知识也不是规则



每向右一步，结构化和责任要求增加；最终规则与 SOP 必须有适用范围、版本、复审和回滚。

检索治理：相关不等于适用

检索检查
服务/集群范围
时间有效期
权限
证据来源
冲突

为什么需要
避免跨环境混用经验
Runbook 和规则会过期
不同角色可见信息不同
相关文档不等于事故证据
多个版本/规则可能互相矛盾

失败时怎么做
缩小 **scope** 或转人工
标记 **stale**，要求复审
脱敏、拒绝或请求授权
把知识与实时观测分栏
提示冲突，不自动执行

工具注册表：让能力可发现、可评估、可淘汰



- ▶ 每个工具有适用对象、权限、风险、超时、失败语义和验证器；
- ▶ 记录调用次数、成功率、耗时、结果质量和当前版本；
- ▶ 软件系统变化后，工具可以灰度、降级或退役。

工具图与 Scoreboard: 能力资产的可视化



- ▶ Tool Graph: 工具之间的依赖与顺序;
- ▶ Scoreboard: 质量、耗时、成功和适用环境;
- ▶ Tool Library: 描述、版本、元数据和淘汰。

把工具当作平台资产管理, 才能减少团队重复写脚本和“只有某个专家会用”的隐性风险。

来源: OpsLens 系统原型独立工具图。

工具评分表：不只看能不能跑

Approach	Telecom					Bank					Market				
	C@1	C@3	P@1	P@3	S _R	C@1	C@3	P@1	P@3	S _R	C@1	C@3	P@1	P@3	S _R
None-LLM-Based Methods															
MicroRCA [42]	–	–	33.33	50.00	0.1093	7.19	14.29	21.43	42.86	0.2691	–	–	20.83	37.50	0.1492
MicroRank [49]	–	–	16.67	33.33	0.1100	5.36	10.71	19.64	39.29	0.2395	–	–	22.92	29.17	0.1179
MicroScope [24]	–	–	16.67	50.00	0.1933	8.93	12.50	25.00	48.21	0.2705	–	–	19.57	23.91	0.0833
TraceAnomaly [25]	–	–	16.67	50.00	0.1650	7.14	17.86	21.43	35.71	0.2439	–	–	26.42	32.08	0.1336
Nezha [50]	7.14	21.43	28.57	57.14	0.3686	7.32	10.98	21.95	43.90	0.2488	0.00	6.58	23.68	36.84	0.1818
LLM-Based Methods															
RCA-agent [46]	27.45	–	50.98	–	0.3722	20.59	–	42.65	–	0.3063	4.73	–	29.73	–	0.1548
OpsLens w/o MAS	21.57	33.33	43.14	56.86	0.4410	15.44	38.24	41.18	66.18	0.5159	12.84	21.62	51.35	66.22	0.4376
OpsLens w/o R	31.37	41.18	54.90	70.59	0.5455	38.24	47.79	60.29	70.59	0.5974	10.14	13.51	33.11	41.22	0.2634
OpsLens w/o P	45.10	52.94	68.63	74.51	0.6473	53.68	67.65	74.26	84.56	0.7690	22.97	31.76	52.70	64.19	0.4747
OpsLens	56.86	64.71	80.39	86.27	0.7616	60.29	79.41	81.62	94.12	0.8812	37.84	51.35	77.03	91.89	0.7366

- ▶ 输入是否覆盖目标故障；
- ▶ 输出是否结构化、可解释、可复核；
- ▶ 结果是否稳定、耗时是否可接受；
- ▶ 系统变更后是否仍适用。

来源：OpsLens 系统原型独立工具图与评分表示意。

workflow库：把“场景经验”变成可配置产品

workflow元素	平台化形式	例子
触发	事件、定时、工单、变更	每日巡检、P1 告警
步骤	状态、工具、规则、Agent	查询指标、生成摘要
门禁	权限、风险、审批、时间窗	生产写操作
验证	系统、业务、回归检查	503 恢复、下单成功率
反馈	人审、结果、规则、版本	规则灰度和复审

平台边界：哪些能力应该统一，哪些必须留在域内？

平台统一

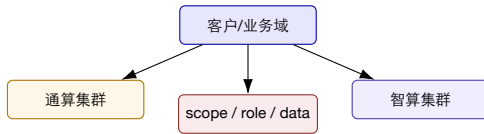
- ▶ 事件 ID、对象目录、Trace、权限和审计；
- ▶ 工具注册、工作流编排、评测和版本；
- ▶ 共享的基础知识和治理策略。

业务域保留

- ▶ 业务语义、SLO、领域规则和数据权限；
- ▶ 高风险动作的负责人和审批逻辑；
- ▶ 域内特殊的验证器与补偿流程。

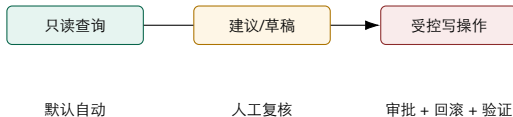
原则 平台统一“共同语言和治理”，业务域保留“语义、责任和风险”。

多租户与多集群：上下文隔离是安全功能



- ▶ 隔离键决定工具端点、指标字典、Skill 范围和可见知识；
- ▶ 诊断结论必须同时声明 **scope**、置信度、限制和下一步；
- ▶ 跨集群证据默认不可混用，除非有明确关联规则。

安全分区：查询、建议、执行三条通道



安全架构要防止“聊天窗口—自由文本—生产命令”的隐式直连。

平台如何证明自己真的在变好？

价值

MTTR / MTBF

质量

证据覆盖

安全

越权动作

成本

调用与人力

采用

接管与复用

平台指标必须按场景、风险和团队分层；不要用一个总分掩盖高风险失败。

第四部分小结：平台提供共同语言和控制面

1. 用对象、时间、语义和血缘把数据连接起来；
2. 用知识、规则和 SOP 将经验结构化；
3. 用 Tool Registry、工作流库和评测资产支持复用；
4. 用隔离、权限、审批和审计控制跨团队风险；
5. 平台统一治理，业务域保留语义和责任。

五、组织与持续运营

没有新的工作方式，平台就会回到旧的人工流程

组织转型：四类角色缺一不可

平台团队 数据接入、工具、工作流、权限和稳定性。

业务 **SRE** 服务语义、SLO、Runbook、接管和复盘。

AI/数据团队 模型、检索、评测、数据质量和成本。

治理/安全 风险等级、审批、审计、合规和供应链。

关键变化 平台团队不替业务团队承担业务判断；业务团队也不应各自维护一套不可复用的智能化基础设施。

CoE：把局部试点变成组织学习



- ▶ CoE 不应变成所有事故的中央值班队伍；
- ▶ 它负责共同语言、参考实现、评测集、风险门和经验传播；
- ▶ 业务团队负责把场景真正接入日常工作。

平台团队与业务团队：共同交付，而不是互相甩锅

对象	平台团队负责	业务团队负责
数据	接入、模型、质量、权限	业务语义、SLO、Owner 和有效标签
工具	契约、版本、运行、审计	领域逻辑、验证器和风险边界
workflow	编排、状态、复用、评测	流程责任、审批人、接管和复盘
结果	平台可靠性、成本和安全	业务恢复、用户影响和改进优先级

治理委员会要做的不是审批每一次查询

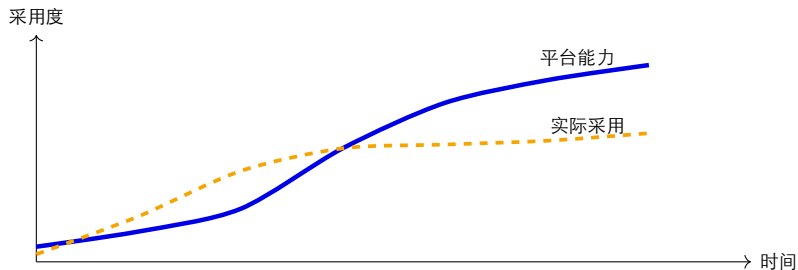
定规则 风险分级、数据分类、工具白名单和审批策略。

看趋势 质量、安全、成本、采用、例外和复发。

做决策 继续、收窄、灰度、暂停、替换或退役。

好的治理是“把常见低风险动作标准化，把例外和高风险动作显式升级”。

采用曲线：技术可用不等于组织愿意用



培训、接管设计、反馈和激励决定两条曲线能否靠近。

运维人员需要学习什么？

读证据 时间窗、
来源、采样、冲突
和未知。

用工具 参数、权
限、失败语义、幂
等和验证。

接管 **Agent** 如何
复核假设、批准动
作、停止和回滚。

做复盘 把高质量
轨迹转成规则、
Runbook 和测试。

能力迁移 AI 原生运维不是降低工程要求，而是把工程判断从命令层提升到证据、策略和系统层。

持续运营节奏：日、周、月、季度

周期	例行动作	产出
日	失败任务、异常数据、未接管事件	当日问题清单和修复责任
周	规则/Prompt/工具回归、成本和延迟	版本变更与灰度计划
月	场景收益、安全例外、采用和反馈	继续/收窄/扩展决策
季度	资产盘点、数据保留、权限复审、演练	能力成熟度和预算调整

预算：把模型成本和人力成本放在同一张表

Token

模型与 Embedding

Tool

查询、存储、执行

People

建设、复核、培训、接管

- ▶ 低延迟场景可能需要小模型、缓存或预计算；
- ▶ 高风险场景的人审和复盘不是“额外成本”，而是控制成本；
- ▶ 用“每个成功任务的端到端成本”而非单次 API 价格衡量。

知识和规则的生命周期



没有退役机制的知识库，会把旧规则和新系统的错误一起召回。

复盘：把一次失败变成四类资产

数据 补充标签、
时间线、缺失源。

知识 Runbook、
规则、案例和限制。

测试 回放样本、
故障注入和验证器。

组织 责任、培训、
流程和阶段门。

复盘标准 复盘不是把故事写得更完整，而是让下一次任务更快、更安全、更可比较。

风险登记：转型要管理“不会做”和“做错了”

风险	早期信号	控制措施
数据不完整	查询经常 partial、时间错位	质量门、降级为只读、补采集
知识过期	引用旧版本 Runbook	TTL、版本和复审队列
错误自动化	工具调用参数异常	schema、dry-run、审批、回滚
采用不足	值班人员绕过平台	改善证据视图、培训和流程集成
成本失控	Token/工具调用快速上涨	缓存、模型路由、预算和限流

第五部分小结：长期运营决定转型成败

1. 平台团队、业务 SRE、AI/数据团队和治理团队共同负责；
2. 采用度需要培训、接管和复盘机制支撑；
3. 知识、工具、模型和工作流都需要版本、灰度、退役；
4. 预算要同时看模型、工具和人的控制成本。

六、综合案例与转型评审

用阶段门回答：这项能力能不能从试点进入生产？

研讨任务：为一个大型客户选择首个场景

1. 从每日巡检、告警治理、变更前检查、RCA 草稿中选一个；
2. 写出价值基线和主要风险；
3. 列出最少需要的对象、数据、工具和知识；
4. 画出人机接管点、验证器和停止条件；
5. 设计 4 周试点与最终阶段门。

小组产出：一页场景章程、一张工作流、一张验收表。

案例 A：电商客户的四周路线

周次	工作	通过标准
第 1 周	盘点缓存、熔断、队列、调用链和变更数据	对象/时间/Owner 可关联
第 2 周	构造缓存热 Key 故障演练和回放样本	有注入真值、清理脚本和恢复验证
第 3 周	上线只读预警与 RCA 草稿	证据链完整，值班人可复核
第 4 周	对照大促演练，比较人工与智能辅助	MTTR、证据、成本、安全均达标

停止条件 演练无法清理、恢复验证不通过、误报导致值班绕过平台，立即收窄范围。

案例 A：转型评审要看哪些数字？

MTTR

是否更快

证据

是否可复核

演练

是否提前发现

安全

是否越权

采用

是否接管

“53 分钟变 5 分钟” 只有在相同故障、相同证据窗口和清晰人工基线下才有意义。

案例 B：金融客户的告警治理

现状抽象

- ▶ 历史系统与云原生系统并存；
- ▶ 告警、工单、应用和基础设施数据分散；
- ▶ 核心账务动作风险高，审批严格。

首轮目标

- ▶ 统一事件身份和责任映射；
- ▶ 只读摘要和影响面分析；
- ▶ 规则复审和人工确认，不自动改账务系统。

金融场景的“低风险”往往意味着只读和建议，而不是减少审计。

案例 B：阶段门评审

阶段门	合格证据	不合格处理
身份	告警、客户、服务、账务域关联正确	停止自动摘要，先补数据契约
摘要	引用正确规则和实时证据	进入人工草稿模式
接管	负责人能在工单中复核并确认	收窄到单一团队/班次
运营	规则有复审、成本和审计	暂停扩展，修复治理流程

案例 C：通信客户的自然语言运维入口

1. 用户请求：“检查华南核心集群最近 10 分钟异常，并给出处置建议”；
2. Agent 解析区域、集群、时间窗、权限和业务对象；
3. 平台调用指标、日志、Trace、拓扑和历史规则；
4. 返回事件摘要、证据链接、候选原因和下一步；
5. 写操作必须进入变更单、审批、执行和验证。

能力边界 自然语言降低入口门槛，但不能绕过权限、范围和动作门禁。

案例 C：平台采用看板

区域	看什么	为什么
入口	请求解析成功率、范围修正次数	用户是否能正确表达目标
取证	工具命中、部分失败、查询耗时	平台是否真的减少找数据
判断	证据覆盖、冲突、人工采纳	结论是否可信
执行	审批、回滚、验证、越权	是否安全参与生产
采用	日活、复用、绕过平台、培训反馈	是否形成新工作方式

大型客户转型评审：五个必须回答的问题

1. 业务损失或工程瓶颈是否被基线证明？
2. 数据、知识和工具是否足以支撑当前范围？
3. 人机责任和高风险动作是否清楚？
4. 端到端结果是否同时改善质量、安全、成本和采用？
5. 新场景接入是否复用平台，而不是重写一套系统？

评审结论 继续扩展、收窄范围、补数据、改流程、转仿真或停止，都比“继续堆模型”更专业。

课堂评分量规

维度	4 分表现	常见失分
价值	有真实基线、业务 Owner 和目标指标	只写“提高效率”
场景	范围、风险、数据和接管明确	把全公司都列入首轮
平台	能力可复用、接口有契约、资产有版本	每个团队各写一套脚本
治理	有权限、审批、回滚、审计和复审	自动化绕过责任链
运营	有反馈、预算、培训、阶段门和停止条件	只交 Demo，不交运营计划

转型路线的一页版



每一步都要留下可回放资产；没有资产沉淀，就只能重复做项目。

本讲总结：从工具建设到能力体系

1. 大型客户的难点是系统复杂度叠加数据、流程、组织和责任断点；
2. 能力演进从手工、监控、AIOps 到 LLMOps/AgentOps，同时提高治理要求；
3. 场景选择要平衡价值、风险、数据和接管成本；
4. 平台应统一共同语言、工具、流程、权限、审计和评测，业务域保留语义与责任；
5. 转型成功的证据是可量化收益、可控风险、可复用资产和持续采用。

带走一句话 企业智能运维转型不是“把 Agent 接到监控上”，而是把发现、判断、处置、验证和学习重构成一条可运营的责任链。

下一讲预告：AIOps 平台架构与工程实践

平台 数据流、能力分层、组件边界。

工程 监控、告警、自动化、配置、知识和处置如何协同。

案例 结合开源蓝鲸和企业平台工程化实践。

第 10 讲回答“为什么转、如何组织”；第 11 讲开始回答“平台具体怎么建”。

参考材料

1. 脱敏行业案例、AgenticSRE、OpsLoop 与 OpsEval 系统原型；
2. 《Enterprise Trends for Generative AI》，Google Cloud Next '24，本地课程材料；
3. 'agentworkflows.pdf'：Agents for Enterprise Workflows；
4. 'Overview.pdf'、'ai-agents.pdf'：企业数据、工具、记忆、安全与 Agent 落地背景；
5. OpsLens、WeRCA、AlertGuardian、AgentTracer 原始论文；
6. 第 9 讲《智能体融入运维工作流》及其材料分析文档；
7. 中国信通院《云计算智能化运维（AIOps）能力成熟度模型》。

脱敏案例数字、界面和架构均标注证据类型；不将单次案例外推为通用生产保证。

课后反思

1. 如果你是客户 CTO，首个试点会选什么，为什么不选另一个？
2. 如果数据质量只达到 70%，系统应自动降级到什么模式？
3. 试点失败时，哪些资产可以保留，哪些设计必须推倒？
4. 如何把“采用度”变成平台工程团队的正式目标？

开放问题 真正的转型能力，不是把所有问题都自动化，而是知道哪些问题值得自动化、怎样安全地自动化。

谢谢

问题、案例与转型路线评审